근거로 설계하고, 실험으로 증명하다

고령층 복지정책 RAG 챗봇

데이터 수집·전처리 / 시스템 아키텍처 / RAG 구현 / 테스트 결과

PROJECT TEAM
소속 SKN36기 · 3rd Team Project
팀명 Team 3
멤버 신지수 · 장선아 · 안홍진 · 김재훈 · 임성경

노인 복지 AI 챗봇 : 실버톡

어르신의 일상적인 표현을 공식 복지정책 근거와 연결해, 필요한 조건과 신청 방법을 쉽게 안내합니다.

어르신의 질문은 보통 이렇게 시작됩니다

“다리가 불편해요.”

정확한 제도명을 몰라도

현재 겪는 어려움을 말하는 것만으로
질문을 시작할 수 있어야 합니다.

우리 챗봇의 3단계 안내 방식

01 **질문 의도 이해**
이동·돌봄·의료 등 어떤 도움이 필요한지 확인

02 **공식 근거 탐색**
관련 복지정책·지침에서 지원 조건 검색

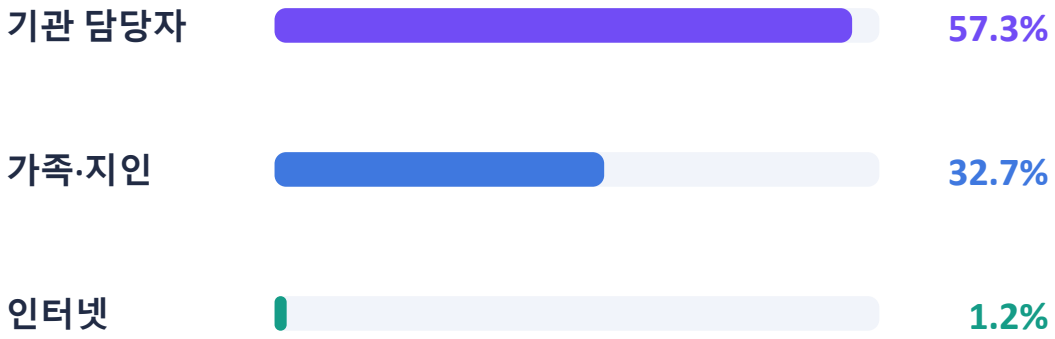
03 **쉬운 말로 안내**
지원 내용·신청 방법·확인할 사항 정리

시장조사: 노인분들은 어디서 복지정책을 확인할까?

① 정보를 어디서 얻는가 → ② 제도를 이해하고 신청할 수 있는가

A 어디에서 알게 될까?

노인일자리 참여자 · 전국 60~74세



기관 + 지인 = 90.0% | 대면·인적 경로가 중심

B 왜 신청하지 못할까?

기초생활보장 미신청자 조사 · 고령층만의 통계 아님

44.8% 제도를 잘 몰라서

22.9% 절차가 복잡해서

두 이유를 합하면 67.7%

SOURCE 한국노인인력개발원(2024) · KIHASA(2023, 손병돈 외 2019 재인용)

시장조사: 고령층의 AI 이용과 장벽

낮은 이용 경험과 사용법·개인정보·절차에 대한 부담을 구분해 살펴봐야 합니다.

서울시 55세 이상

생성형 AI 이용 경험

12.2%

이용 경험이 적다는 사실은 확인되지만,

그 원인을 하나로 단정할 수는 없습니다.

이용 방법을 몰라 AI를 쓰지 않는 경우

미이용자 중 응답 비율 · 전국 별도 조사

60대	25.3%
70세 이상	26.1%

AI 이용을 망설이게 하는 세 가지 진입 부담

01 **사용법** “전문지식이 필요할 것 같아요.”

02 **신뢰** “개인정보가 유출될까 걱정돼요.”

03 **절차** “사용 과정이 복잡할 것 같아요.”

SOURCE 서울AI재단(2025) · KISDI 한국미디어패널(2025) · 방통위·KISDI(2024)

문제정의 및 PainPoints



핵심 원칙 | 정확히 묻지 못해도 도움의 종류를 함께 좁혀 가고, 위험 상황에서는 안전을 우선합니다.

경쟁사 분석 및 우리의 차별성

AI정부24·AI 국민비서는 자연어 검색과 행정서비스 연결을 이미 지원합니다.

공통 기반 일상어 질문 이해 · 관련 복지정보 탐색 · 공식 신청 안내

그렇다면 우리 팀은 어디에 더 집중했을까?

01 상황 표현

“다리가 불편해요.”

정책명을 몰라도
지원 분야를 탐색

의도 파악

02 모호한 표현

“배고파요.”

무료 식사 / 식료품 /
식사 준비 지원 중 선택

선택형 후속 질문

03 위험 표현

“죽고 싶어요.”

복지 검색보다 안전 확인
전문 상담·긴급 도움 안내

안전 우선

차별화 가설 | 답을 한 번에 단정하기보다, 사용자의 의도를 함께 확인해 필요한 도움으로 연결합니다.

기존 서비스와 우리 챗봇, 여섯 가지 사용자 상황을 비교했습니다

팀이 확인한 일부 응답 시나리오 기준입니다. ○ 표시는 실제 검증 범위와 구현·설계 범위를 구분하여 해석해야 합니다.

○ **해당 시나리오에서 지원 확인/설계** △ 일부 지원 · 추가 확인 필요 × 해당 테스트에서 미확인

구분	비교 항목	AI정부24 · AI 국민비서	우리 챗봇
Pain Point 1	정책명을 몰라도 일상어로 질문	○	○
Pain Point 2	고령층 눈높이의 쉬운 설명·단계별 안내	×	○
Pain Point 3	지역·지원 조건 확인 및 공식 근거 안내	△	○
Pain Point 4	모호한 생활 표현을 선택형 질문으로 구체화	×	○
Pain Point 5	위험 표현 감지 및 별도 안전 안내	×	○
추가 기능 1	질문 의도를 파악해 사용자 상황에 맞게 풀어 설명	△	○

※ ×는 서비스 전체의 기능 부재가 아니라 팀의 제한된 테스트에서 미확인을 뜻하며, 우리 챗봇의 ○도 실서비스 검증 완료와는 구분합니다.

01

시스템 아키텍처

- ① 수집 → 검색 → 답변의 흐름
- ② LLM과 데이터베이스의 분명한 책임 분리
- ③ 보안·개인정보 / ④ 장애 대응·확장

SKN36 / TEAM 3

노인복지 챗봇 전체 아키텍처

공식 복지자료를 검색할 수 있게 준비하고, 질문이 들어오면 근거와 함께 답변합니다.

01 데이터 선별 및 적재

질문을 받기 전에 준비

데이터 수집
지자체 복지서비스 API
보건복지부 공식 PDF 4종

→

선별·전처리
노인 관련 정책 선별 · PDF 섹션화
문서 정리 후 800자 청킹 · 150자 중첩

→

문서 임베딩
KURE-v1 · 1,024차원
청크 3,400개 벡터화

→

데이터 적재
Qdrant · 벡터·본문·출처 정보
Neo4j · 벡터·청크와 원본 관계

Neo4j 출처 경로 API 청크 → 복지정책 | PDF 청크 → 섹션 → 원본 PDF

적재된 자료를 검색에 활용

02 RAG 및 UI 연결

질문 입력
React 채팅 화면
질문 · 선택 지역 · 세션 전달

→

대화 처리
FastAPI → LangGraph
안전·의도 확인 후 검색 여부 결정

→

근거 검색
KURE-v1으로 질문 임베딩
Qdrant 의미 검색 → Neo4j 출처 조회

→

답변 표시
근거 확인 → OpenAI LLM 답변
FastAPI → React에서 답변·출처 표시

PostgreSQL 대화 기록 저장·조회

질문이 모호하거나 근거가 부족하면 추가 질문 또는 확인 안내

02

수집된 데이터 및 데이터 전처리 문서

- ① 출처와 수집 범위가 명확한가
- ② 중복·오류·누락을 어떻게 처리했는가
- ③ 검색에 적합하게 문서와 메타데이터를 설계했는가

DATA PIPELINE

데이터 수집 및 활용

챗봇 답변의 근거는 지자체 복지서비스 API와 보건복지부 공식 PDF, 두 축으로 구성했습니다.

01 지자체 복지서비스 API

지역별 복지서비스 목록 **1,485건** → 서비스 ID 기반 상세 조회 **1,000건** → 노인복지 관련 정책 선별 **574건**

출처: 공공데이터포털

활용: 지역별로 어떤 지원이 있는지 찾는 데 사용

02 보건복지부 공식 PDF



1,189개 섹션으로 정리

- 기초연금 사업안내
- 노인보건복지사업안내
- 기타 보건복지부 공식 지침

활용: 제도의 세부 기준과 신청 절차 설명에 사용

정리 포인트

- 지자체 데이터 = 지역별 지원 탐색
- 공식 PDF = 기준·절차 설명

→ 즉, 저희 챗봇은 **'무슨 지원이 있는지'**와 **'어떻게 신청하는지'**를 함께 설명합니다.

PREPROCESSING & INDEXING

데이터 전처리 및 적재

API와 PDF는 형식이 다르기 때문에 문서화, 청킹, 임베딩, 저장 단계를 거쳐 검색 가능한 구조로 만들었습니다.

원천 데이터 형식

API 원천

정책명 · 지원 대상 · 신청 방법
항목별 구조화 데이터

정책 1건 = 문서 1개

PDF 원천

장 · 절 · 본문 · 표 중심의
비정형 문서

섹션별로 목차 + 본문 정리

데이터 전처리 및 적재 프로세스 (4단계)

01 문서 정리

API 문서화
PDF 섹션화 +
페이지 정보 유지

02 청킹

800자 단위
150자 중첩

API	662개
PDF	2,738개
총	3,400개

03 임베딩

KURE-v1

모든 청크를
동일 모델로 벡터화

04 저장

Qdrant

벡터 + 원문 +
출처 정보

Neo4j

청크와 상위 자료를
관계로 연결

Qdrant에 저장한 정보

- 청크별 벡터
- 원문
- 출처 정보

Neo4j 관계 구조

API 청크 → 복지정책

PDF 청크 → 섹션 → 원본 PDF

청크 하나를 찾으면, 어떤 정책·공식문서에서 나온 내용인지 추적할 수 있습니다.

참고: 청킹 방식과 임베딩 모델을 선택한 이유는 뒤의 테스트 결과에서 설명합니다.

03

RAG 기반 LLM과 벡터 DB 연동 구현 코드

- ① 임베딩·저장·검색·생성 정상 동작
- ② 답변의 근거와 출처
- ③ 예외 처리·비밀키 / ④ 구조와 실행 가능성

01 실제 사용자 질문이 답변이 되기까지

하나의 질문을 따라가며 RAG 질의응답 처리의 실행 흐름을 확인

USER QUESTION **“마포구에서 보행 보조기구 지원받을 수 있나요?”**

01
질문 접수
자연어 질문과
대화 맥락 전달

→

02
질문 분석
의도·지역 정보 확인
검색 조건 구성

→

03
정책 검색
Qdrant 의미 검색
Neo4j 관계 조회

→

04
답변 생성
정책 원문을 근거로
LLM 응답 생성

→

05
결과 반환
쉬운 정책 설명과
출처·확인 사항 제공

이 페이지의 핵심 **설계도 나열이 아닌, 한 질문이 검색 → 근거 확인 → 응답까지 이동하는 실제 처리 과정**

02 Qdrant로 의미가 비슷한 복지정책 찾기

정확한 정책명을 몰라도 문장의 의미를 벡터로 표현해 유사한 근거를 탐색

SEARCH PIPELINE

"마포구에서 보행 보조기구 지원돼요?"

↓

KURE-v1 · 1,024차원

↓

Qdrant · Cosine Similarity Top-5

실제 검색에서 확인한 예

저소득 노인 보행보조차 지원

parent_id local:WLF00000696

- 정책 고유 ID를 보존해 원본 및 관계 탐색에 활용
- 키워드가 달라도 의미가 유사한 문서를 검색

구현상 주의 · 의미적 관련성 ≠ 지역별 실제 수급 가능성 | 서비스 검색 경로의 지역 필터 통합 검증 필요

03 Qdrant + Neo4j, 검색 근거에 문맥 더하기

벡터 검색으로 찾은 청크를 문서 ID로 연결하여 원문과 관계 정보를 추적

01 QDRANT

의미 기반 후보 탐색

- 질문과 유사한 정책·PDF 청크 Top-K
- 본문과 chunk_id / parent_id 반환

ID 연계

02 NEO4J

문서 구조·연결 관계 탐색

- 청크 → 섹션 → 원본 문서 관계 조회
- 연관 정책 후보와 상위 근거 추적

HYBRID RESULT · 근거 본문 + 문서 관계 정보

정책 사실의 근거 **Qdrant 원문** · 관계·추천 후보는 Neo4j에서 보조적으로 활용 (확정 사실과 구분)

04 검색 결과를 “근거 있는 답변”으로 변환

실제 정책 본문만 사실 근거로 삼고, 검증되지 않은 조건은 단정하지 않도록 제한

01 근거 정리

빈 본문·중복·ID 불일치 제외
실제 근거와 추천 후보 구분

→

02 LLM 답변 생성

정책 본문을 Context로 전달
ChatOpenAI로 쉬운 한국어 설명

→

03 답변 + 출처 반환

answer 및 sources 반환
미확인 정보는 추가 확인 안내

ANSWER GUARDRAILS

- 문서에 없는 지원 금액·신청 조건·URL을 임의 생성하지 않음
- 지역·소득·수급 자격을 임의로 확정하지 않음
- 충분한 근거가 없으면 답변 대신 확인 필요 안내

응답 구성 예시

answer	정책 설명 · 확인 필요 사항
sources	정책명 · 원본 문서 · 출처

검증 범위 · sources 반환 구조는 확인됨 / 문장별 출처 일치·정확한 인용 페이지 자동 검증은 후속 과제

PILLAR 04 / QUALITY ASSURANCE

04

테스트 계획 및 결과 보고서

- ① 기능·검색·답변 검증 범위
- ② 정답률·근거·속도의 평가 지표
- ③ 조건과 결과의 재현성 / ④ 발견 문제와 개선

01 테스트 계획 | 9개 검증 항목으로 서비스 평가

데이터 정확성부터 검색·API·답변 신뢰성·운영 안정성까지 영역별로 구분

ID	테스트 항목	평가 목적 / 내용
T01	데이터 무결성	ID·원본 참조 일관성
T02	청킹·임베딩 비교	Fixed / Structure × KURE / Qwen
T03	검색 경로 A/B/C	Qdrant / Neo4j / 결합 검색
T04	DB·출처 연결	적재 수량·PDF 관계 검사
T05	질문 API	입력 검증·선택지 응답
T06	근거 부족 대응	기권·출처 처리
T07	세션 기억	이력 조회·저장 제한
T08	최종 답변 평가	자격·금액·근거 충실도
T09	운영 안정성	503·실서비스 부하·보안

집중 분석: T02 청킹·임베딩 선택 → T03 Qdrant·Neo4j 검색 경로 비교

02 T02 | 청킹 방식 × 임베딩 모델 비교

동일한 원본·질문·검색 조건으로 문서 분할 방식과 임베딩 모델의 영향을 검증

CHUNKING

문서를 어떻게 나누는가?

A_fixed

800자 기준 분할

150자 Overlap

B_structure

항목·문단 경계

의미 단위 우선

청크 수 **A: 3,400개 · B: 3,865개**

EMBEDDING

텍스트의 의미를 어떻게 표현하는가?

KURE-v1

한국어 추가 학습

1,024차원

Qwen3 0.6B

비교 임베딩 모델

1,024차원

통제 조건 **113개 채점 질문 · 지역 필터 · Top-5**

실험 목적 **직관적으로 좋은 분할보다, 실제 정답 근거를 더 잘 찾아주는 조합을 선택**

03 평가 지표 | 검색 성능은 어떻게 측정했나?

테스트 결과에 앞서 네 가지 검색 평가지표의 의미를 간단히 정리

HIT@1 첫 번째 검색 결과 적중률

검색 결과 1위에 정답 근거가 포함된 질문의 비율

HIT@5 상위 5개 검색 결과 적중률

상위 5개 중 정답 근거가 하나라도 포함된 질문의 비율

RECALL@5 정답 근거 회수율

찾아야 할 정답 근거 중 상위 5개에서 실제로 찾아낸 비율

MRR@5 첫 정답의 평균 순위 점수

첫 정답이 앞에 나올수록 높아지는 역순위의 평균값

다음 장에서 동일한 113개 질문에 적용한 네 조합의 실제 검색 결과를 비교

04 T02 결과 | A_fixed + KURE-v1이 가장 우수

같은 113문항으로 네 조합을 비교했을 때 검색 적중률과 첫 정답 순위에서 우위

	A_fixed	B_structure
KURE-v1	88.5% BEST MRR 0.765	85.8% MRR 0.713
Qwen3 0.6B	77.0% MRR 0.606	77.9% MRR 0.596

핵심 결과 A_fixed + KURE-v1 · Hit@5 88.5% · Recall@5 85.0% · MRR@5 0.765

※ 2×2 비교는 검색 모델 선택을 위한 실험이며, 응답의 사실 정확성은 별도 검증 필요

05 T03 | Qdrant·Neo4j 검색 A/B/C 비교

단독 검색 두 방식과 순위 결합 검색을 비교해 추가 성능 이점과 시간 비용 확인

A **Qdrant 단독**
Vector Search

B **Neo4j 단독**
Vector Search

C **A+B 결합**
RRF Rank Fusion

검색 품질

A = B = C

v3 단일 질문	Hit@5 89.4% · MRR@5 0.769
ABC 복수 근거	Recall@10 0.929

v3 검색 시간



해석 173개 실제 검색 질문의 반환 순위가 동일 → 이번 조건에서 결합 검색의 순위 이점 없음, 검색 시간은 증가

APPENDIX | 검증 가능한 원본 증빙 경로

실제 파일과 저장된 실험 결과를 바로 열어볼 수 있도록 핵심 링크를 정리

데이터 출처·전처리	elderly_filter.py	**Qdrant v1 적재**	04_qdrant_load_v1.ipynb
PDF 검증	step06_validation_report.json	**Neo4j·GraphRAG**	hybrid_retriever.py
청크·ID 검증	step07_chunking_report.json	**LangGraph 실행**	graph.py
A/B 최종 실험 결과	summary.csv	**FastAPI**	main.py
실험 조건·해시	run_meta.json	**개인정보·대화 저장**	session_manager.py
함정 질문 결과	trap.csv	**환경·DB 실행**	docker-compose.yml